

在有些情况中,被引用的存储器位置实际上是存储在磁盘存储器上的。此时,硬件会产生一个缺页(page fault)异常信号。同其他异常一样,这个异常会导致处理器调用操作系统的异常处理程序代码。然后这段代码会发起一个从磁盘到主存的传送操作。一旦完成,操作系统会返回到原来的程序,而导致缺页的指令会被重新执行。这次,存储器引用将成功,虽然可能会导致高速缓存不命中。让硬件调用操作系统例程,然后操作系统例程又会将控制返回给硬件,这就使得硬件和系统软件在处理缺页时能协同工作。因为访问磁盘需要数百万个时钟周期,OS缺页中断处理程序执行的处理所需的几百个时钟周期对性能的影响可以忽略不计。

从处理器的角度来看,将用暂停来处理短时间的高速缓存不命中和用异常处理来处理长时间的缺页结合起来,能够顾及到存储器访问时由于存储器层次结构引起的所有不可预测性。

#### 旁注 当前的微处理器设计

一个五阶段流水线,例如已经讲过的 PIPE 处理器,代表了 20 世纪 80 年代中期的处理器设计水平。Berkeley 的 Patterson 研究组开发的 RISC 处理器原型是第一个 SPARC 处理器的基础,它是 Sun Microsystems 在 1987 年开发的。Stanford 的 Hennessy 研究组开发的处理器由 MIPS Technologies(一个由 Hennessy 成立的公司)在 1986 年商业化了。这两种处理器都使用的是五阶段流水线。Intel 的 i486 处理器用的也是五阶段流水线,只不过阶段之间的职责划分不太一样,它有两个译码阶段和一个合并的执行/访存阶段[27]。

这些流水线化的设计的吞吐量都限制在最多一个时钟周期一条指令。4.5.9 小节中描述的 CPI(Cycles Per Instruction, 每指令周期)测量值不可能小于 1.0。不同的阶段一次只能处理一条指令。较新的处理器支持超标量(superscalar)操作,意味着它们通过并行地取指、译码和执行多条指令,可以实现小于 1.0 的 CPI。当超标量处理器已经广泛使用时,性能测量标准已经从 CPI 转化成了它的倒数——每周期执行指令的平均数,即 IPC。对超标量处理器来说,IPC 可以大于 1.0。最先进的设计使用了一种称为乱序(out-of-order)执行的技术来并行地执行多条指令,执行的顺序也可能完全不同于它们在程序中出现的顺序,但是保留了顺序 ISA 模型蕴含的整体行为。作为对程序优化的讨论的一部分,我们将会在第 5 章中讨论这种形式的执行。

不过,流水线化的处理器并不只有传统的用途。现在出售的大部分处理器都用在嵌入式系统中,控制着汽车运行、消费产品,以及其他一些系统用户不能直接看到处理器的设备。在这些应用中,与性能较高的模型相比,流水线化的处理器的简单性(比如说像我们在本章中讨论的这样)会降低成本和功耗需求。

最近,随着多核处理器受到追捧,有些人声称通过在一个芯片上集成许多简单的处理器,比使用少量更复杂的处理器能获得更多的整体计算能力。这种策略有时被称为“多核”处理器[10]。

## 4.6 小结

我们已经看到,指令集体系结构,即 ISA,在处理器行为(就指令集合及其编码而言)和如何实现处理器之间提供了一层抽象。ISA 提供了程序执行的一种顺序说明,也就是一条指令执行完了,下一条指令才会开始。

从 IA32 指令开始,大大简化数据类型、地址模式和指令编码,我们定义了 Y86-64 指令集。得到的

ISA 既有 RISC 指令集的属性, 也有 CISC 指令集的属性。然后, 将不同指令组织放到五个阶段中处理, 在此, 根据被执行的指令的不同, 每个阶段中的操作也不相同。据此, 我们构造了 SEQ 处理器, 其中每个时钟周期执行一条指令, 它会通过所有五个阶段。

流水线化通过让不同的阶段并行操作, 改进了系统的吞吐量性能。在任意一个给定的时刻, 多条指令被不同的阶段处理。在引入这种并行性的过程中, 我们必须非常小心, 以提供与程序的顺序执行相同的程序级行为。通过重新调整 SEQ 各个部分的顺序, 引入流水线, 我们得到 SEQ+, 接着添加流水线寄存器, 创建出 PIPE-流水线。然后, 添加了转发逻辑, 加速了将结果从一条指令发送到另一条指令, 从而提高了流水线的性能。有几种特殊情况需要额外的流水线控制逻辑来暂停或取消一些流水线阶段。

我们的设计中包括了一些基本的异常处理机制, 在此, 保证只有到异常指令之前的指令会影响程序员可见的状态。实现完整的异常处理远比此更具挑战性。在采用了更深流水线和更多并行性的系统中, 要想正确处理异常就更加复杂了。

在本章中, 我们学习了有关处理器设计的几个重要经验:

- 管理复杂性是首要问题。想要优化使用硬件资源, 在最小的成本下获得最大的性能。为了实现这个目的, 我们创建了一个非常简单而一致的框架, 来处理所有不同的指令类型。有了这个框架, 就能够在处理不同指令类型的逻辑中共享硬件单元。
- 我们不需要直接实现 ISA。ISA 的直接实现意味着一个顺序的设计。为了获得更高的性能, 我们想运用硬件能力以同时执行许多操作, 这就导致要使用流水线化的设计。通过仔细的设计和分析, 我们能够处理各种流水线冒险, 因此运行一个程序的整体效果, 同用 ISA 模型获得的效果完全一致。
- 硬件设计人员必须非常谨慎小心。一旦芯片被制造出来, 就几乎不可能改正任何错误了。一开始就使设计正确是非常重要的。这就意味着要仔细地分析各种指令类型和组合, 甚至于那些看上去没有意义的情况, 例如弹出值到栈指针。必须用系统的模拟测试程序彻底地测试设计。在开发 PIPE 的控制逻辑中, 我们的设计有个细微的错误, 只有通过控制组合的仔细而系统的分析才能发现。

### 网络旁注 ARCH:HCL Y86-64 处理器的 HCL 描述

本章已经介绍几个简单的逻辑设计, 以及 Y86-64 处理器 SEQ 和 PIPE 的控制逻辑的部分 HCL 代码。我们提供了 HCL 语言的文档和这两个处理器的控制逻辑的完整 HCL 描述。这些描述每个都只需要 5~7 页 HCL 代码, 完整地研究它们是很值得的。

## Y86-64 模拟器

本章的实验资料包括 SEQ 和 PIPE 处理器的模拟器。每个模拟器都有两个版本:

- GUI(图形用户界面)版本在图形窗口中显示内存、程序代码以及处理器状态。它提供了一种方式简便地查看指令如何通过处理器。控制面板还允许你交互式地重新启动、单步或运行模拟器。
- 文本版本运行的是相同的模拟器, 但是它显示信息的唯一方式是打印到终端上。对调试来讲, 这个版本不是很有用, 但是它允许处理器的自动测试。

这些模拟器的控制逻辑是通过将逻辑块的 HCL 声明翻译成 C 代码产生的。然后, 编译这些代码并与模拟代码的其他部分进行链接。这样的结合使得你可以用这些模拟器测试原始设计的各种变种。提供测试脚本, 它们全面地测试各种指令以及各种冒险的可能性。

## 参考文献说明

对于那些有兴趣更多地学习逻辑设计的人来说, Katz 的逻辑设计教科书[58]是标准的人门教材, 它强调了硬件描述语言的使用。Hennessy 和 Patterson 的计算机体系结构教科书[46]覆盖了处理器设计的广泛内容, 包括这里讲述的简单流水线, 还有并行执行更多指令的更高级的处理器。Shriver 和 Smith [101]详细介绍了 AMD 制造的与 Intel 兼容的 IA32 处理器。